Build log

A running record of what has been added to the project: what each piece is, how it works inside, what the names and variables in the code mean, why it was chosen that way and which C#, PostgreSQL or tooling idea it teaches. Read it once from start to finish, then update it with every pull request.

Versión en español: BUILD-LOG-ES.md. Exported PDF/Word files live in `exports/`.

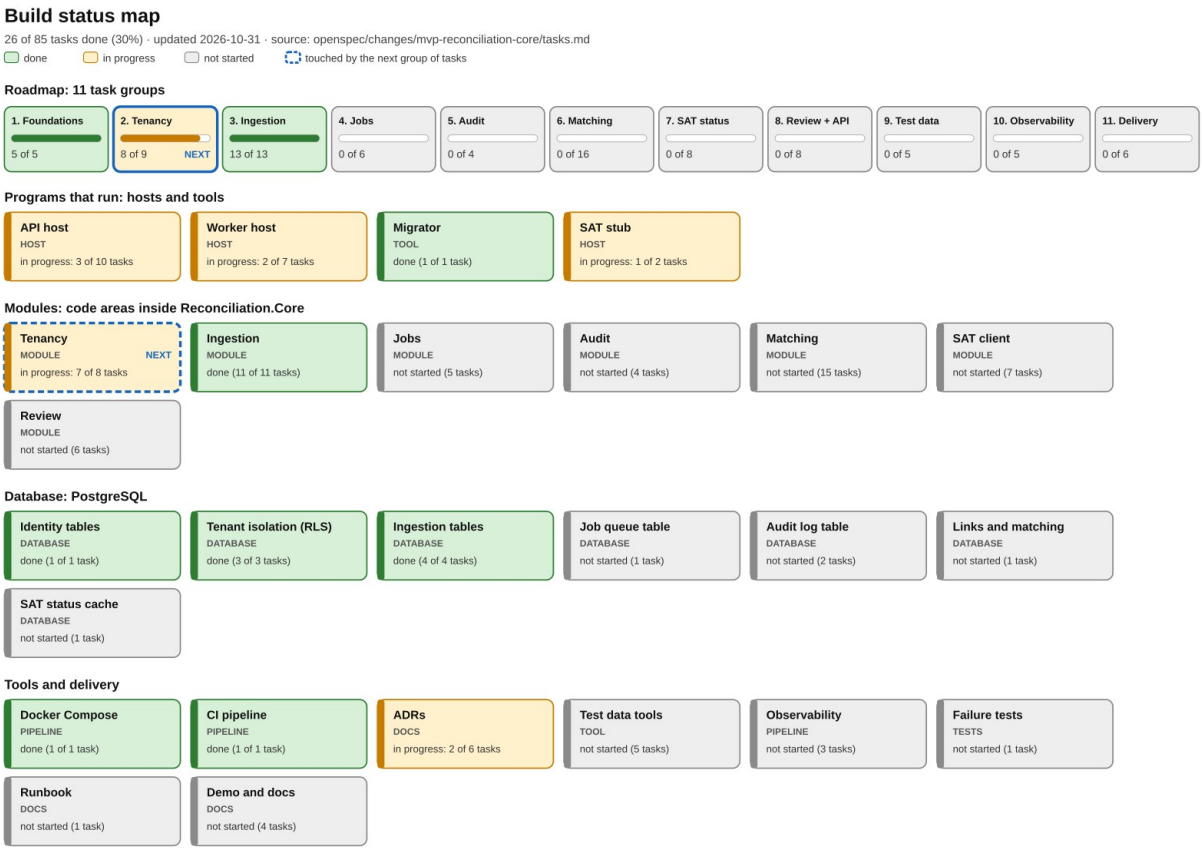
Source of truth for the work items: `openspec/changes/mvp-reconciliation-core/tasks.md` (85 tasks in 11 groups). This log explains them; the status map below is generated from them.

How the documentation is organised

Folder	What it holds	Why it is separate
`docs/adr/`	Architecture decision records (ADR-001 to ADR-004)	One decision = one file; they are never edited, a later ADR replaces them
`docs/build-log/`	This log in English and Spanish, and `exports/` with the generated PDF/Word files	What changes every PR; exports stay apart from the source text
`docs/requirements/`	BRS and SRS (English and Spanish)	Requirements documents that rarely change
`docs/reference/`	The reference bank CSV format and the project brief with reading links	Reference material that tests read or that is cited
`docs/diagrams/`	Database and role diagrams and the progress map (`progress/`)	Drawn or generated images, each with its `.svg` source
`openspec/`	The specification and tasks of the `mvp-reconciliation-core` change	Source of truth for scope

Rule of thumb: before deleting or moving a file in `docs/`, search for its name with `grep -rn` (several xUnit tests read `docs/adr/`, `docs/reference/` and `docs/build-log/`), and use `git mv` to keep history.

1. Status at a glance



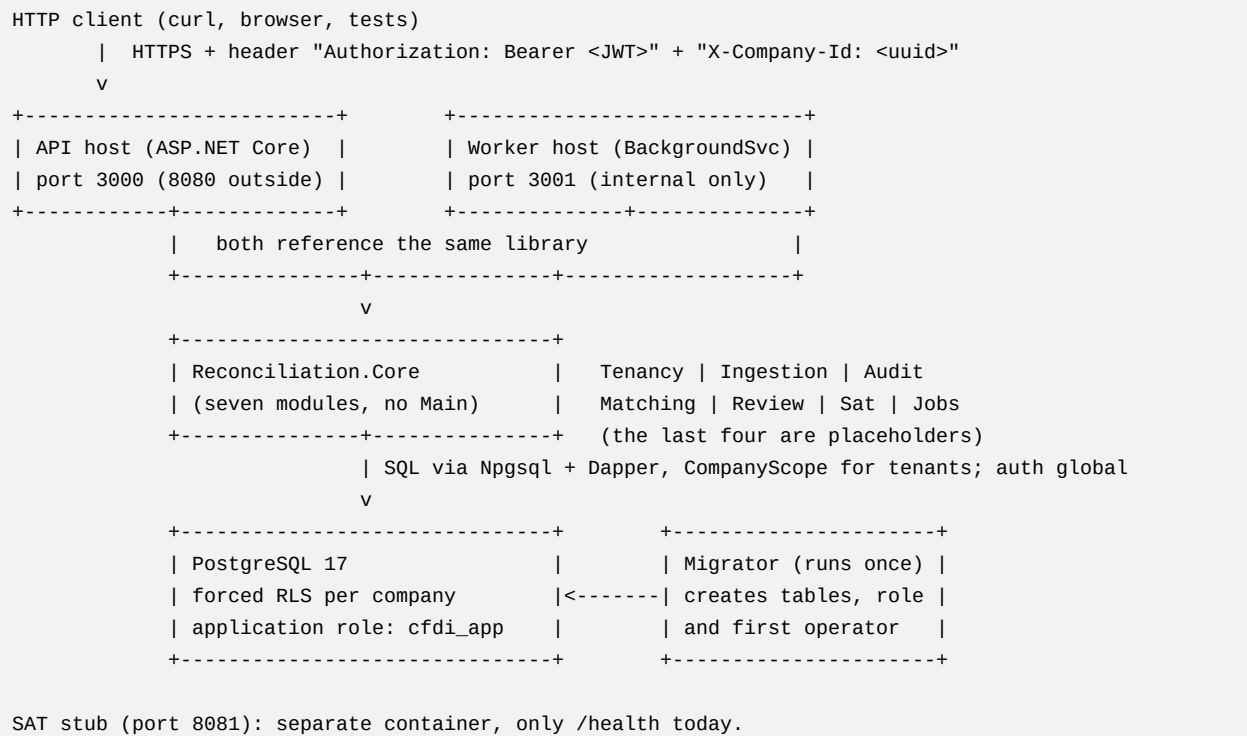
26 of 85 tasks complete. Group 3 is complete; in group 2 only 2.4 remains, waiting for real group 8 decisions. Regenerate the image after ticking tasks: `python3 tools/progress_diagram.py` (add `--lang es` for Spanish).

What each type means

Type	What it is	Example here
Host	A program that keeps running and serves requests or does background work	API, worker, SAT stub
Module	An area of code inside the shared library with one business responsibility	Matching, Audit
Database	Tables, rules and security that live inside PostgreSQL	`company`, row-level security
Tool	A program that runs once to do a job and exits	Migrator, test-data generators
Pipeline	Automation that builds, tests or starts things	Docker Compose, CI
Docs / Tests	Written decisions, or code that checks the rest of the code	ADRs, xUnit tests

2. Architecture

2.1 Overview: what is built today

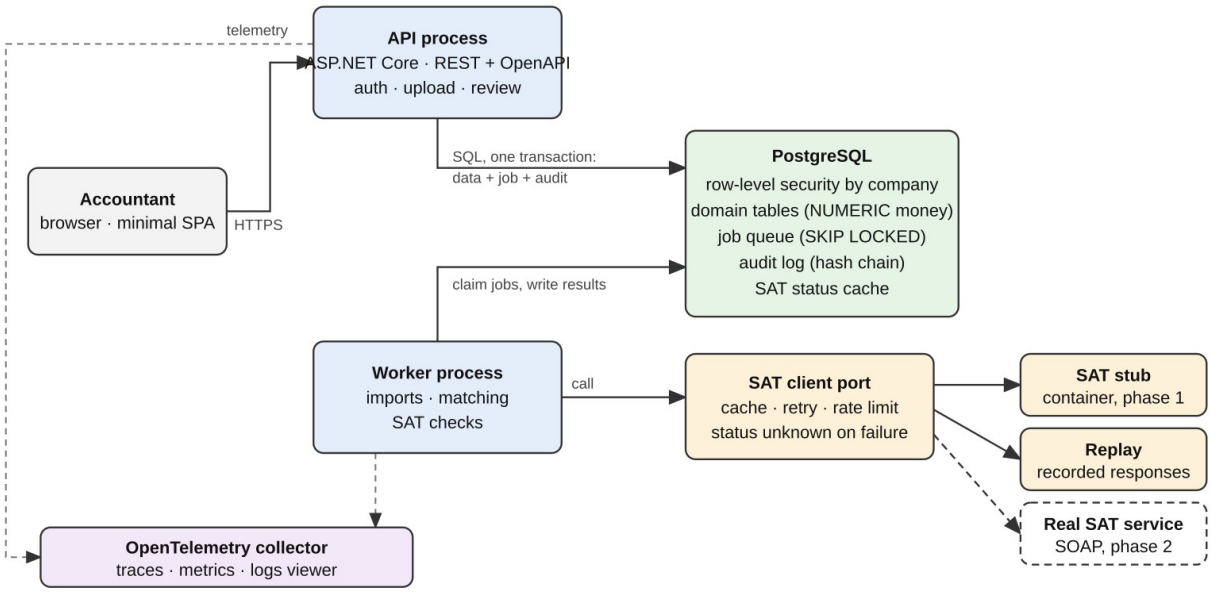


Every arrow is a boundary with a reason:

- **Two hosts, one library.** The API serves people and must answer quickly; the worker will do long tasks (matching, SAT queries). They share `Reconciliation.Core`, so business rules are written once (ADR-001: modular monolith with a separate worker, instead of microservices).
- **Modules with boundaries.** `Tenancy` (companies, users, roles, sessions), `Ingestion` (reading CFDI and CSV), `Audit` (log), and the placeholders `Matching`, `Review`, `Sat`, `Jobs` that groups 4 to 8 will fill. A module talks to another through interfaces, never by reading its tables.
- **PostgreSQL as the final referee.** Even if the code forgets to filter by company, the database does not return another company's rows (section 2.3).
- **The migrator is a separate program** with a different database role: the schema owner (`cfdi`) creates and changes tables; the application (`cfdi_app`) can only do what it is granted.

The full design, including what does not exist yet (job queue, SAT client, telemetry), is this:

Container view (Docker Compose)



Dashed line: optional or telemetry flow. Only the SAT stub and the SAT client port differ between phase 1 and phase 2.

What exists so far is the *API process* box, the *Worker* (it only starts and reports health), *PostgreSQL* with company isolation and the ingestion tables, and the *SAT stub* as a placeholder. Still missing: the job queue (`SKIP LOCKED`), the audit hash chain, the SAT client port and OpenTelemetry.

2.2 Life of an authenticated request

Example: an accountant uploads an invoice with `POST /companies/{companyId}/imports/cfdi`.

1. **Login.** The client calls `POST /auth/tokens` with `{"email","password"}`. Before touching the database, `LoginProtection` reads the body (8 KiB maximum), normalises the email (trim and upper-case) and applies two fixed-window limits: 10 attempts per minute per email and 30 per minute per IP; beyond them it answers 429 with `Retry-After`. It then finds the user case-insensitively and performs **exactly one** PBKDF2 verification: against the real hash or, if the email does not exist, against a dummy hash (`DummyPasswordHash`) created once at start-up, so response time does not reveal which emails exist. On a match it returns a signed JWT (HMAC-SHA256) valid for 15 minutes with `sub`, `sid` and `jti` claims.
2. **Request.** The client sends the file with `Authorization: Bearer <token>` and `X-Company-Id: <uuid>`.
3. **Authentication.** ASP.NET validates signature, issuer, audience and expiry. If that fails the answer is 401.
4. **Per-company authorisation.** `CompanyRoleHandler` requires `X-Company-Id` to match the route `{companyId}`, opens a transaction with that context and reads the user's role from `membership`. The `accountant` policy accepts administrator and accountant; a `viewer` gets 403.

5. **Company context.** `CompanyScope.ExecuteAsync` opens a connection, starts a transaction and runs `set_config('app.company_id', <uuid>, true)`; the `true` makes it last only for that transaction. For write operations, in the same command batch (`NpgsqlBatch`) it takes a lock on the company row (`SELECT archived_at IS NOT NULL FROM company FOR UPDATE`): if the company is archived it throws `CompanyReadOnlyException` and the API answers 409; the lock also serialises simultaneous changes (for example two administrators being demoted at the same time).
6. **Work.** The importer reads and validates the file and inserts rows; every query goes through the RLS policies.
7. **Close.** If everything worked, `COMMIT`; if anything fails, `ROLLBACK` and nothing is left half done. The response is an `ImportReport` with one result per file. An unexpected error goes through `UseExceptionHandler` and is returned as `ProblemDetails` with a safe message and no internal detail; if some files were already committed, the error includes the `uploadId` so the partial report can be fetched.

2.2a Session renewal

`add-session-renewal` is tracked independently from the 85 MVP tasks. It adds no company-data route and does not change their Bearer contract. The API issues a 15-minute access JWT and a 256-bit base64url refresh credential; only SHA-256 `token_hash` is stored. `sub` identifies the user, `sid` the family and `jti` the access token. `SessionService` uses injected `TimeProvider` for issuance, validation, deadlines and events. `SessionOptions` validates at startup. `CompanyRoleHandler` still reads membership live; roles stay outside tokens and sessions.

1. Login verifies the slow salted password hash and inserts a family, refresh digest and `login` event in one transaction.
2. Renewal looks up the digest, locks `auth_session` with `SELECT ... FOR UPDATE`, re-reads the token, and rotates it atomically with its `refresh` event. It updates `last_used_at`; the new expiry is the lesser of idle time and the original cap.
3. A replay through the inclusive grace boundary returns 409 and changes nothing. Later replay revokes the family and writes `reuse_detected`; other devices remain active. Clients coordinate with single-flight renewal.
4. Logout revokes the owned family. Password change verifies the current password, hashes the new one and revokes all of that user's sessions with `password_changed`, atomically. Auth events are append-only for `cfdi_app`.
5. Access stays stateless until expiry; no session lookup is added to company requests. Optional HttpOnly/Secure/ SameSite=Strict cookies use Path=/auth and require an allowed Origin for renewal; default transport remains JSON. Login keeps its email/IP limits; refresh and logout share a named IP limit. See ADR-004.

2.3 Security layers (defence in depth)

Layer	What it stops	Where it lives	How it is tested
Login attempt limit	Brute force and credential stuffing	`LoginProtection.cs`	`ReviewHardeningTests`
JWT authentication	Anonymous requests	`Authentication.cs`, fallback policy	`AuthenticationTests`

Layer	What it stops	Where it lives	How it is tested
Membership and role per company	Using a company where you have no role, or an action the role cannot do	`CompanyRoleHandler.cs`	`MembershipTests`, `IsolationTests`
Transaction with context	Queries with no company assigned	`CompanyScope.cs`	`CompanyScopeTests`
Archived company is read-only	Writing to an archived company, or losing the last administrator through a race	`CompanyScope.cs` (row lock)	`ReviewHardeningTests`
Forced RLS in PostgreSQL	Seeing or writing another company's rows even if a `WHERE` is missing	Migrations `M0002` and `M0004`	Isolation guard over every table and route
Composite foreign keys	A row pointing at another company's record	`FOREIGN KEY (company_id, x_id)`	`IngestionDatabaseTests`
Unprivileged role	Bypassing RLS, deleting data	`cfdi_app`: no superuser, no `BYPASSRLS`, no `DELETE`	Tests that use the real role
Safe XML reading	XXE attacks and "bomb" files	`SafeCfdiXmlReader.cs`	`SafeXmlTests`
Input limits	Oversized files or CSVs; hostile file names	`IngestionOptions`, `MultipartUpload.cs`	`ReviewHardeningTests`, `IngestionHttpTests`
Safe downloads and errors	The browser interpreting an original; leaking internal detail	`Content-Disposition: attachment`, `nosniff`, `ProblemDetails`	`ReviewHardeningTests`

Why several layers: each covers a different failure. A 403 does not prove the RLS policy exists, and an RLS policy does not identify the user; both are needed, and each needs tests.

2.4 Life of an import

```
multipart (files, bank, accountReference)
  -> MultipartUpload: counts bytes while reading, cleans file names (413/400)
  -> per file: SafeCfdiXmlReader (one bounded parse) -> CfdiParser -> is it the company's?
  -> per file, ITS OWN transaction:
       import_file in state 'importing' (metadata only, private to the transaction)
       -> rows in batches (CFDI, payments, relations; movements in chunks of 5000)
       -> finalised once: original + validated hash -> 'imported' -> audit
  -> ImportReport { upload_id, files[ status, reason, ids ] }
```

The key point is "one transaction per file": if the third of ten files is invalid, the other nine are imported and the third is stored as `rejected` with its reason. All files of one upload share an `upload_id` (a new UUID per upload) so the full report can be requested later. The transient `importing` row is never committed: because it lives only inside the transaction, the bytes of the original (up to 10 MiB) are written and verified once, instead of re-hashing every time a counter is updated.

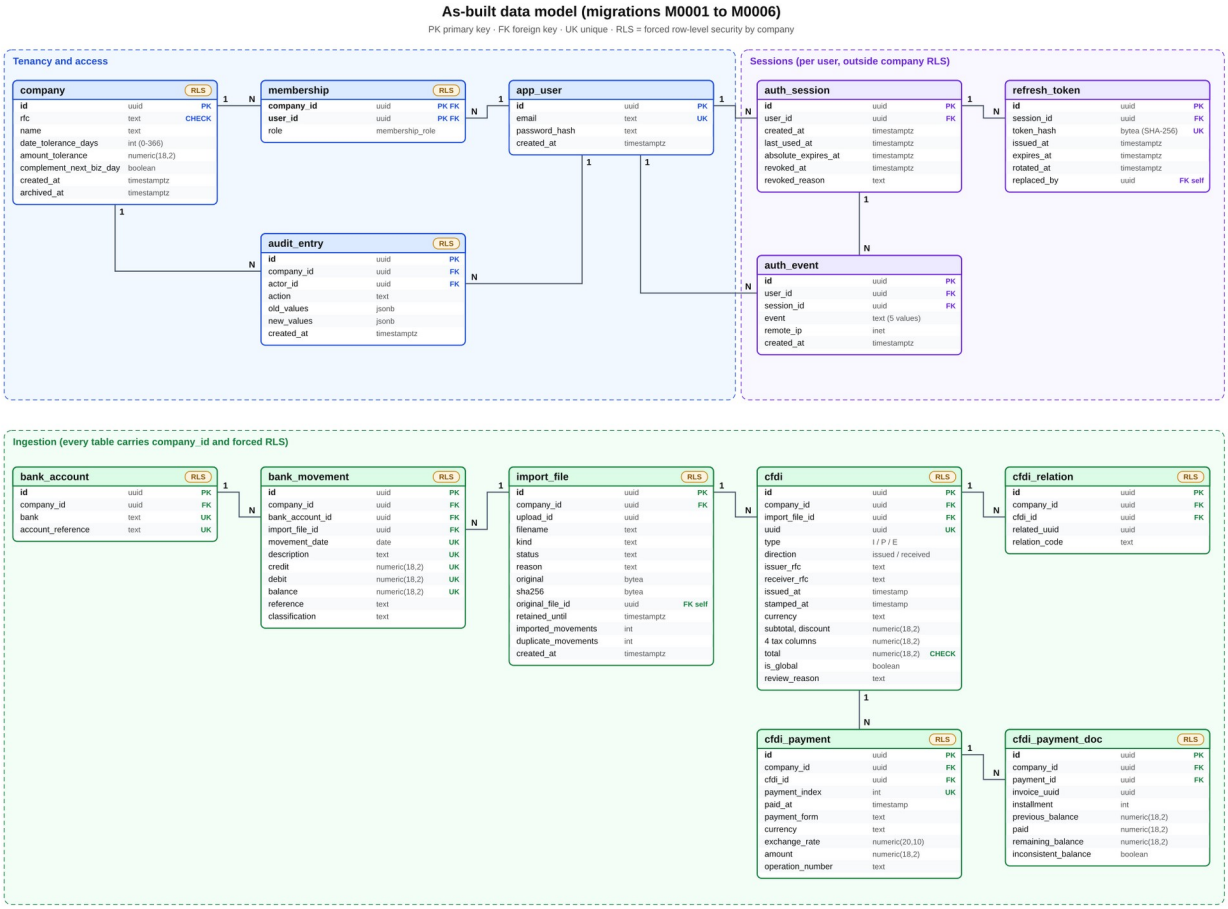
2.5 Data model

Table	Why it exists	Key columns
`company`	Each customer (company) in the system	`rfc` (immutable, validated), `date_tolerance_days` (0–366), `amount_tolerance`, `archived_at`
`app_user`	People who log in	`email` (unique, case-insensitive), `password_hash`
`membership`	Links user, company and role	PK `(company_id, user_id)`, `role` (`administrator`, `accountant`, `viewer`)
`audit_entry`	Who did what and when	`actor_id`, `action`, `old_values`, `new_values` (jsonb)
`import_file`	An uploaded file and its outcome	`upload_id`, `status`, `reason`, `original` (bytes), `sha256`, `retained_until`, `original_file_id`
`cfdi`	An imported invoice	`uuid`, `type` (I/P/E), `direction`, amounts `numeric(18,2)`, `is_global`, `review_reason`
`cfdi_relation`	Relations between CFDIs (for example a credit note)	`related_uuid`, `relation_code`
`cfdi_payment`, `cfdi_payment_doc`	Payment complement and the invoices it settles	`payment_index`, `installment`, `previous_balance`, `paid`, `remaining_balance`
`bank_account`	A bank account identified by a label	`bank`, `account_reference`
`bank_movement`	One line of the bank statement	`movement_date`, `credit`, `debit`, `balance`, `reference`

Conventions repeated in every company table: `id uuid` generated by the database; mandatory `company_id`; RLS enabled

and forced; composite foreign keys; money in `numeric(18,2)`, never `double` or `float`; indexes that start with `company_id` (`M0005`) so each company query walks only its own rows.

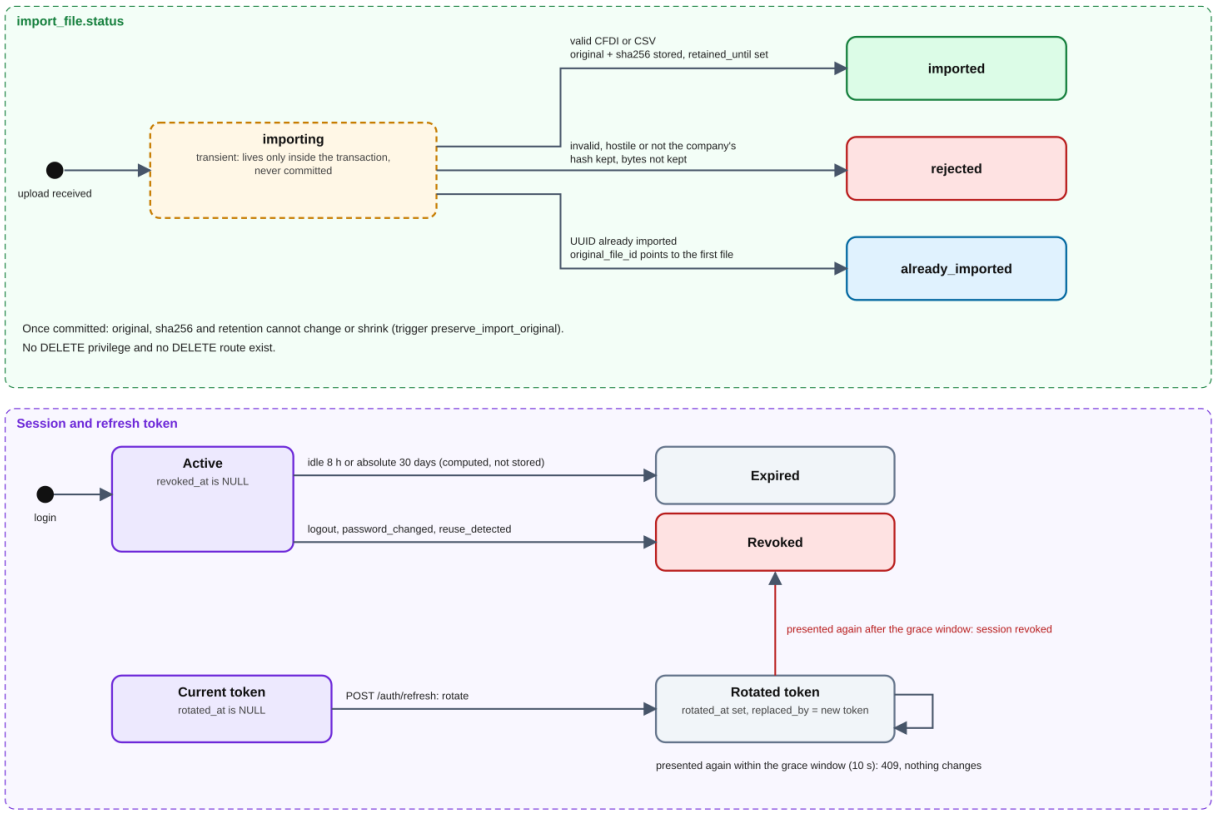
The as-built schema (migrations M0001 to M0006), drawn from the migrations themselves:



Blue is tenancy, purple is sessions (per user, outside company RLS) and green is ingestion; the orange RLS badge marks tables with forced row-level security. The next two diagrams show the status rules of the data and who may do what:

Status and lifecycle rules of the data

import_file.status · auth_session / refresh_token



Row security, privileges and immutability

S = SELECT · I = INSERT · U(columns) = UPDATE only on the named columns · no table grants DELETE

Table	Area	Forced RLS	Privileges of role cfdi_app	Immutability / triggers
company	tenancy	yes	S I U	rfc cannot change (trigger company_rfc_immutable)
membership	tenancy	yes	S I U	
app_user	tenancy	no (per user)	S I U(password_hash)	
audit_entry	tenancy	yes	S I	append-only
auth_session	sessions	no (per user)	S I U(last_used_at, revoked_at, revoked_reason)	
refresh_token	sessions	no (per user)	S I U(rotated_at, replaced_by)	
auth_event	sessions	no (per user)	S I	append-only
import_file	ingestion	yes	S I U(status, reason, original, sha256, ...)	original, sha256 and retention frozen (trigger)
cfdi	ingestion	yes	S I U(review_reason)	
cfdi_relation	ingestion	yes	S I	
cfdi_payment	ingestion	yes	S I	
cfdi_payment_doc	ingestion	yes	S I U(inconsistent_balance)	
bank_account	ingestion	yes	S I	
bank_movement	ingestion	yes	S I	

The original design-time model (planned tables for the whole MVP) is still in `docs/diagrams/database/database-erd.png`. Regenerate the three as-built diagrams with `python3 tools/db_diagrams.py` (add `--lang es` for Spanish).

2.6 Per-user authentication storage

The company/ingestion diagrams describe their tenant model. M0006 adds these global per-user tables outside company RLS; they have no `company_id` and do not use `CompanyScope`. Existing company policies and SQL names stay unchanged.

Table	Fields and purpose
`auth_session`	`id`, `user_id`, `created_at`, `last_used_at`, `absolute_expires_at`, `revoked_at`, `revoked_reason`: one device/family
`refresh_token`	`id`, `session_id`, `token_hash`, `issued_at`, `expires_at`, `rotated_at`, `replaced_by`: immutable digest/deadline and rotation link
`auth_event`	`id`, `user_id`, `session_id`, `event`, `remote_ip`, `created_at`: login/refresh/reuse/logout/password events, no credentials

Indexes cover the unique digest, family and user. `cfdi_app` has SELECT/INSERT, UPDATE only on named mutable columns, no DELETE, and no event UPDATE. `app_user` gains only UPDATE(password_hash). Down removes the new tables and that grant. All session timestamps are explicit UTC values from TimeProvider. Group 5 will integrate events into chained auditing; expired-row purging waits for jobs. Existing access can outlive revocation until its configured expiry.

3. Dictionary of names and variables

3.1 Environment variables and configuration

Name	Example value	Read by	What it means
`ConnectionStrings__Default`	`Host=postgres;Port=5432;Username=cfdi_app;...`	API, worker, migrator	PostgreSQL connection string. The double underscore is how .NET writes `ConnectionStrings:Default` in an environment variable. API and worker use role `cfdi_app`; the migrator uses owner `cfdi`
`APP_DB_PASSWORD`	synthetic placeholder	Migrator and Compose	Password the migrator uses to enable login for role `cfdi_app` (`ALTER ROLE ... LOGIN PASSWORD`)
`Jwt__SigningKey` / `JWT_SIGNING_KEY`	at least 32 random bytes	API	Secret key used to sign and verify tokens. If missing, short or equal to the old public sample value, the API will not start. Compose **requires** `JWT_SIGNING_KEY` (there is no default any more) and passes it to the API as `Jwt__SigningKey`; generate one with `openssl rand -hex 32`
`OPERATOR_EMAIL`, `OPERATOR_PASSWORD`	optional; password of 12+ characters	Migrator (`up`)	If present, the platform operator is created once; an existing operator's password is never reset (it only logs that the account was not modified)
`Ingestion__MaxFileBytes`	10485760 (10 MiB)	Ingestion	Maximum size of one file. A larger one rejects only that file
`Ingestion__MaxUploadBytes`	20971520 (20 MiB)	Ingestion	Maximum size of the whole upload, counting multipart headers. Exceeding it rejects the entire upload (413)

Name	Example value	Read by	What it means
`Ingestion__MaxFiles`	20	Ingestion	Maximum number of files per upload (400 if exceeded)
`Ingestion__MaxXmlNodes`	100000	Ingestion	Maximum XML nodes and attributes; stops "bomb" files
`Ingestion__MaxXmlDepth`	64	Ingestion	Maximum XML nesting depth
`Ingestion__MaxStatementLines`	100000	Ingestion	Maximum rows in a bank statement; exceeding it rejects that whole file with its reason
`Ingestion__MaxDescriptionLength`	500	Ingestion	Maximum characters of the description; bounds the size of the unique-index keys of movements
`Ingestion__RetentionYears`	10 (minimum 5, maximum 100)	Ingestion	Years the original is kept; sets `retained_until`
`LoginRateLimit__EmailPermitLimit`	10	API	Login attempts per normalised email in each fixed window
`LoginRateLimit__IpPermitLimit`	30	API	Login attempts per remote IP in each fixed window
`LoginRateLimit__WindowSeconds`	60	API	Window length; exceeding a limit returns 429 with `Retry-After`
`RUN_BENCHMARKS`	`1` (only when measuring)	Tests	Enables the opt-in performance benchmark; without it (and in CI) it is skipped

`IngestionOptions.Validate()` and `LoginProtection` check everything at start-up: a non-positive value, a retention outside 5–100 or a non-positive login window stops the program from starting, instead of failing later in the middle of an operation. The per-email limit can be abused to throttle a victim (an attacker exhausts their attempts); the default is generous and the IP limit helps, but production needs tuning and monitoring.

3.1a Session dictionary and configuration

Name	Default / meaning
`Session__AccessTokenMinutes`	15; allowed 5–30; returned as `expiresIn` seconds
`Session__IdleMinutes`	480; renewal inactivity deadline
`Session__AbsoluteDays`	30; cap fixed at login, never extended
`Session__ReuseGraceSeconds`	10; inclusive replay conflict window
`Session__RefreshCookie`	false; optional cookie instead of JSON refresh delivery
`Session__AllowedOrigins__0`	allowed HTTPS frontend Origin, e.g. https://app.example.invalid; required for successful cookie renewal
`RefreshRateLimit__IpPermitLimit`	60 per LoginRateLimit__WindowSeconds, shared by refresh/logout per IP
`SessionOptions`, `TimeProvider`	validated configuration and injected clock; tests use ManualTimeProvider without sleeps

Name	Default / meaning
`accessToken`, `refreshToken`, `expiresIn`	login/renewal response; refreshToken omitted in cookie mode
`sid`, `token_hash`, `replaced_by`	family id; SHA-256 of the wire credential; successor token id
`reuse_detected`	revocation/event reason after grace; 409 inside grace changes nothing
`/auth/refresh`, `/auth/logout`, `/auth/password`	renew; end the owned family; change own password and end all own families

3.2 HTTP headers, routes, fields and status codes

Name	Where it appears	Meaning
`Authorization: Bearer <JWT>`	Header	The token obtained at login
`X-Company-Id`	Header	The company the request acts on; must equal the route `{companyId}` and the user must be a member
`{companyId}`	Route	The company UUID
`{uploadId}` / `upload_id`	Route / column	UUID that groups the files of one upload
`{invoiceId}`	Route	Internal UUID (`cfdi.id`) of the invoice. It is not the fiscal UUID
`{fileId}`	Route	UUID of an `import_file` row
`files`	Multipart field	One or more uploaded files; names are cleaned (no control characters, 255 maximum)
`bank`	Multipart field	Bank format identifier; today only `reference`
`accountReference`	Multipart field	A label of 1–40 ASCII letters/digits, spaces, hyphens and underscores. Never a real account number
`Retry-After`	Response header	Seconds to wait after a login 429
`Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`	Response headers	On downloads: force saving the file and forbid guessing its type

Code	When
400	Malformed body or too many files
401	No token, invalid token or wrong credentials
403	Authenticated but without the required role in that company
404	Resource does not exist **or belongs to another company** (deliberately not distinguished)
405	Method not allowed; for example there is no `DELETE`
409	Archived company: read-only; or a rotated refresh credential inside its reuse grace window

Code	When
413	Login body or upload larger than the limit
429	Too many login or session renewal/logout attempts; Retry-After tells when to retry
500	Unexpected error; a safe `ProblemDetails` is returned with no internal detail

3.3 Database names

Name	Meaning
`app.company_id`	PostgreSQL session variable holding the current company. RLS policies compare it with `company_id`
`cfdi` (user)	Schema owner; only the migrator uses it. It can bypass RLS, which is why the application does not use it
`cfdi_app` (role)	The application role: `NOSUPERUSER NOBYPASSRLS`, only `SELECT/INSERT` and `UPDATE` on specific columns
`membership_role`	Enumerated type: `administrator`, `accountant`, `viewer`
`uuid` (in `cfdi`)	The fiscal identifier of the CFDI (the SAT folio). It is unique **per company**
`type`	`I` income, `E` expense (credit note), `P` payment (payment complement)
`direction`	`issued` if the company is the issuer; `received` if it is the receiver
`status` (in `import_file`)	`imported`, `already_imported` (repeated UUID), `rejected`, or `importing` (transient: never left committed)
`original_file_id`	For a duplicate, points to the file that was already accepted
`sha256`	32-byte fingerprint of the file; lets you prove it did not change
`retained_until`	Date before which the original cannot be deleted or shortened
`preserve_import_original`	Trigger: only an `importing` row may set its original (by becoming `imported`); an accepted original, its hash and its retention never change
`date_tolerance_days`	Days of difference allowed between bank and fiscal date when matching (default 3, maximum 366: `company_date_tolerance_max`)
`amount_tolerance`	Amount difference allowed when matching (default 0.01)
`complement_next_biz_day`	Deadline rule for payment complements; used in group 6
`VersionInfo`	FluentMigrator table with the applied versions (today 1 to 6)
`numeric(18,2)`	Money with 18 digits and exactly 2 decimals
`numeric(20,10)`	Exchange rate with up to 10 decimals

3.4 Frequent names in the C# code

Name	Meaning
`actorId`	UUID of the user performing the action; comes from the `sub` claim and is stored in the audit
`uploadId`	UUID generated for each upload (`Guid.NewGuid()`)

Name	Meaning
`UploadedFile(Name, Content, Hash, Error)`	A file in memory with its SHA-256 fingerprint; `Error` carries the reason if it could not be read
`FileImportOutcome`	Result for one file: status, reason, ids and movement counts
`ImportReport(UploadId, Files)`	What the API returns: all results of one upload
`UploadFailedException`	Database error in the middle of an upload; carries the `UploadId` so what was already committed can be read
`ParsedCfdi`, `PaidInvoice`	Already-read representation of a CFDI and of each payment line (the balance flags are immutable)
`InconsistentBalance`	Mark on a payment line whose balance does not fit; it does not reject, it sends to review
`IBankStatementParser`	Contract for reading one bank's CSV; one implementation per bank
`CompanyScope`	Service that runs code inside a transaction with the company assigned
`CompanyReadOnlyException`	Thrown when writing to an archived company; the API turns it into 409
`Policies`, `MembershipRoles`	Constants with the policy names (`company-member`, `administrator`, `accountant`, `platform-operator`) and the role names
`JwtTokenPolicy`, `PasswordPolicy`	Constants: issuer/audience (`reconciliation`), zero clock skew and minimum password length (12); access lifetime comes from `SessionOptions`
`CompanyHeader.Name`	The name of the `X-Company-Id` header in one place
`DummyPasswordHash`	Dummy hash created at start-up to equalise the work when the email does not exist
`LoginProtection`	Login email/IP limits plus the named per-IP policy for `/auth/refresh` and `/auth/logout`
`AuditLog.WriteAsync`	The single function that writes to `audit_entry`; every action uses it
`DapperExtensions`	Helpers that build Dapper commands with explicit parameter aliases
`ImportFileStore`	Stores the original files and their metadata

4. What exists today

Piece	Type	What it is for	Where
Solution file	Build	Groups the six projects so one `dotnet build` builds everything	`Reconciliation.slnx`
Shared build rules	Build	Every project uses the same settings: .NET 10, null checking, warnings are errors	`Directory.Build.props`, `.editorconfig`, `global.json`
Core library	Module area	The code both hosts share: tenancy, ingestion, minimal audit and common database and health code	`src/Reconciliation.Core`
API host	Host	The web API users will call. Serves authentication, companies, members, settings, archive, imports, health and OpenAPI	`src/Reconciliation.Api`
Worker host	Host	Runs background jobs. Today it starts, reports health and waits	`src/Reconciliation.Worker`

Piece	Type	What it is for	Where
Migrator	Tool	Applies or reverts database changes (`up`, `down`) and creates the first operator	`src/Reconciliation.Migrator`
SAT stub	Host	A simulation of the SAT status service to develop without a real account. Today only a placeholder	`src/Reconciliation.SatStub`
PostgreSQL 17	Database	Stores all data with forced RLS per company	`docker-compose.yml`, migrations `M0001` to `M0006`
Docker Compose	Pipeline	Starts database, migrator, API, worker and stub with one command and waits for all to be healthy	`docker-compose.yml`, `Dockerfile`
CI pipeline	Pipeline	On every push: format, build, tests, migrations up/down/up and a full system start	`.github/workflows/ci.yml`
ADRs	Docs	Big decisions and the alternatives that were rejected	`docs/adr/`
Requirements and plan	Docs	BRS, SRS, diagrams and the OpenSpec change that defines the work	`docs/requirements/`, `docs/diagrams/`, `openspec/`
Tests	Tests	190 xUnit cases: 186 passing, 3 pending scenarios and 1 opt-in benchmark; real PostgreSQL	`tests/Reconciliation.Tests`
Progress map	Tool	Draws the status image from `tasks.md`	`tools/progress_diagram.py`

5. Pull request log

Most recent first. Each entry answers six questions: **What it is** (the piece), **How it works** (step by step),

Names and variables (what each new term means), **Why this way** (decision and rejected alternative), **What you learn** (C#, SQL or tooling idea) and **Verified** (how it was checked). Entries before PR-5 use the names the code had then; the PR-5 rename map translates them.

PR-6: Session renewal

What it is: the independent `add-session-renewal` change: short access, rotating refresh families, bounded renewal, logout, password change, append-only authentication events and optional browser cookie transport.

How it works: red tests were committed before each new behavior and followed by feature commits. Real PostgreSQL/Testcontainers, the real M0006 migrator and an injected fake clock verify storage, grants, races and expiry. Rotation locks the family, re-reads the credential and uses a second conditional-update guard. Every event shares its state-change transaction. Live membership remains in CompanyRoleHandler; no ORM, new NuGet package or company RLS policy is added for sessions. Existing Bearer routes and existing SQL/JSON names stay intact.

Names and variables: SessionOptions holds the six session settings; TimeProvider supplies UTC time; auth_session is the family; refresh_token stores only token_hash;

auth_event records user/session/event/IP/time. The JWT sid links access to its family, while jti distinguishes access tokens issued at the same instant. A grace 409 differs from a revoked/unknown/expired 401. Cookie refresh requires an allowed Origin, and its secret never appears in JSON. See the architecture and dictionary above and ADR-004.

Why this way: a rotating 256-bit opaque secret needs only a fast SHA-256 digest. The row lock makes concurrent outcomes explicit; sixteen requests produce one winner and fifteen conflicts. The grace window tolerates tabs but can hide an immediate replay, so clients use single-flight. Stateless access retains the documented revocation latency (15 minutes by default). Optional refresh cookies preserve Bearer clients; server-side cookie sessions were rejected, and OIDC remains the production recommendation. IP budgets affect shared NATs. Chained auditing, expired-row purging, MFA/SSO/recovery and a UI remain outside this change's scope.

What I learned: a token lookup performed before a lock can be stale after waiting; read it again under the family lock. Password verification/session creation and password changes serialize on a user row without blocking foreign-key reads. A real denied event INSERT proves rollback instead of merely asserting transaction syntax. Already-green concurrency, expiry, legacy-Bearer and live-role regressions were committed as verification-only changes with user approval; no artificial failures or production changes were introduced. Test-host mutations prove the auth guard rejects an unregistered route and a refresh endpoint that ignores credentials.

Verified: 190 cases: 186 pass and 4 skip (the existing three group 6/8 scenarios and the opt-in ingestion benchmark). Build has zero warnings/errors; format verification passes. Migrations up/down/up run against a disposable database; cfdi_app cannot delete sessions/tokens or alter events and can update only password_hash on app_user. Compose is healthy. The README curl flow and live rotation/grace/password/logout checks pass without logging credentials. Word/PDF exports are regenerated and rendered for inspection. No MVP task/spec/design was edited; supplied session design/spec artifacts remain unchanged. Commits are local; nothing is pushed.

PR-5: Domain naming and de-duplication

What it is: a clean-up with no behaviour change. The code is rewritten in the project's vocabulary (company, CFDI, upload, file) and repetition is removed, preserving routes, JSON fields, SQL table and column names, report statuses and migration versions.

How it was done: five refactor blocks, each verified with the full suite before moving on:

- **B1:** locals and parameters get domain names.
- **B2:** named constants for policies, roles and money/password/JWT/upload limits; statuses and kinds are mapped explicitly to the strings that already exist in the database.
- **B3:** shared code: one audit write, Dapper commands, claims, transaction results, upload validation and error mapping, and test helpers.
- **B4:** CFDI reading/validation was split from persistence, and a payment's balance flags became immutable.
- **B5:** types and files renamed, with the README, ADR-003 and both logs updated together.

Names and variables new or changed:

Old name	New name	Why
`TenantContext`	`CompanyScope`	The transaction scope belongs to one company
`CompanyManagement`	`CompanyAdminEndpoints`	Company administration routes
`ImportStorage`	`ImportFileStore`	Stores retained upload files
`CompanyAuthorization`	`CompanyRoleHandler`	Authorises the role in the requested company
`PlatformAuthorization`	`PlatformOperatorHandler`	Authorises the platform operator
`IngestionServices`	`IngestionServiceCollectionExtensions`	Describes the dependency-injection registrations
`SafeCfdiXml`	`SafeCfdiXmlReader`	Reads XML with safety limits
`PaymentDocument`	`PaidInvoice`	One invoice paid by a complement
`InvoiceResponse`	`CfdiResponse`	Uses the CFDI domain term
`ExistingInvoice`	`StoredCfdi`	The stored result that enables idempotency
`FileImportResult`	`FileImportOutcome`	The outcome of one uploaded file
`LoginRequest`	`TokenRequest`	Token endpoint input
`LoginUser`	`LoginAccount`	The account used to log in
`M0003CompanyManagement`	`M0003OperatorAndAudit`	Describes operator and audit; version 3 is unchanged
`M0004Ingestion`	`M0004IngestionTables`	Describes the ingestion tables; version 4 is unchanged

Route parameters `{id}` became `{companyId}`. The PR-1 to PR-3 entries of this log keep the names the code had at the time; use the table above to translate them. The new constants (`Policies`, `MembershipRoles`, `JwtTokenPolicy`, `PasswordPolicy`, `CompanyHeader`) are in section 3.4.

Why this way: the name `TenantContext` spoke about architecture ("tenant"), not about the business; `CompanyScope` says what happens: a transaction scoped to one company. Deliberate decisions **not** to change: type visibility (kept as it was), the seven planned modules (they stay as the design's module map, with a "planned" comment) and the XML fiscal terms, which stay as the SAT uses them.

Review finding	Fix or decision
Generic names in locals and parameters	Domain names; explicit Dapper aliases; `cancellationToken` and `transactionCancellationToken`
Repeated policy, status and limit literals	Shared constants; enums mapped to the database strings, which do not change
Repeated SQL, audit, upload and test plumbing	`DapperExtensions`, one `AuditLog.WriteAsync`, one upload handler, transaction results, claims and fixture helpers
Long reading and persistence methods	Named stages, `CfdiInsertRow` and immutable `PaidInvoice`
Ambiguous type names	Table above; code, README, ADR-003 and logs updated together
Narrow visibility / delete empty modules / translate XML terms	Intentionally skipped (see above)

What you learn: renaming a local variable can change an inferred anonymous-object property that Dapper uses as a SQL parameter, which is why explicit aliases are written. Renaming a C# type does not change its JSON members. Transaction callbacks use `transactionCancellationToken` and endpoint methods use `cancellationToken`; the parameters of an `override` keep the base signature's name so the analysers stay quiet. A temporary Roslyn tool helped with the renames and was left outside the repository.

Verified: 126 cases: 122 pass and 4 skipped (three future scenarios and the opt-in benchmark); format clean after every refactor commit. The progress map keeps the same completed tasks: none was ticked for clean-up.

PR-4: Hardening and performance

What it is: fixing the findings of a security, correctness and performance review without changing the public data contracts.

How it was done: red tests first (`ReviewHardeningTests`, `PerformanceRegressionTests`) and a baseline measurement with an opt-in benchmark (`RUN_BENCHMARKS=1`) on real PostgreSQL 17: one warm-up plus three samples, same SDK container and same machine. The baseline was committed **before** the production code changed. Then the fixes were applied. Normal runs and CI skip the benchmark.

Workload	Median before (ms)	Median after (ms)	Improvement
50,000 statement lines	13423.6802	4089.7699	≈ 3.28×
Payment with 300 related documents	65.4256	17.5534	≈ 3.73×

Samples before: 13423.6802 / 13529.5307 / 13336.1982 and 65.4256 / 66.9515 / 63.3336. Samples after: 4230.7872 / 4089.7699 / 4083.4153 and 17.5399 / 17.5534 / 17.9515. These are local elapsed timings, not production latency guarantees, and they already include the new security checks and the indexes.

How each fix works:

Finding	Fix or decision
Hostile file names; partial uploads that cannot be discovered	Control characters and NUL are stripped, invalid Unicode is replaced and the name is capped at 255; if the database fails mid-upload, the error returns the `uploadId` of what was already committed
XML attribute amplification	Attributes count toward the node budget; the XML is parsed **once** through a bounded reader
Writes to an archived company; last-administrator race	A lock on the company row guards every write; an archived company answers 409; two simultaneous demotions leave at least one administrator
Different work for unknown emails	One dummy hash created at start-up; exactly one PBKDF2 verification per well-formed attempt
Login abuse	Two chained fixed windows: normalised email (10/min) and IP (30/min), configurable; 429 with `Retry-After`
Public default JWT key	Compose requires `JWT_SIGNING_KEY` and start-up rejects the old public value; CI supplies a synthetic one explicitly

Finding	Fix or decision
Downloads that can be interpreted and buffered	`attachment` + `nosniff`; bytes are streamed (`SequentialAccess`, `Results.Stream`) while the company transaction stays open
Unbounded CSV and index keys that are too large	`MaxStatementLines` = 100000 and `MaxDescriptionLength` = 500; the whole file is rejected with its reason
Unlimited date tolerance; existing operator	Cap of 366 in the API and in `M0005`; start-up logs that an existing account was not modified
Error leakage; OpenAPI	`AddProblemDetails` + `UseExceptionHandler` give safe errors in every environment; OpenAPI stays anonymous per the spec (the suggestion to require authentication in production was dropped)
Row-by-row writes	5000-row `unnest` chunks for movements; one batch of relations, payments and documents; duplicates within and across chunks keep their counts
Re-hashing the original	A transient metadata-only `importing` row is finalised once with the bytes, hash validation and counters; no `importing` row is ever committed; a failure rolls back all rows
Weak NULL-to-bytes guard	`M0005` allows fixing the original only on the `importing` → `imported` transition; a committed rejected or duplicate original cannot change
Extra round trips for UUID, RFC and context	`INSERT ... ON CONFLICT ... RETURNING` with a `SELECT` fallback for duplicates; the (immutable) RFC is cached for 5 minutes; `NpgsqlBatch` folds context, lock and check together
Company lookups without an index	`M0005` adds company-first indexes on files, payments and documents; `membership` already had its `(company_id, user_id)` primary-key index
Memory copies	Capacity hints, buffer plus explicit length, bytea bound as memory, one XML parse, CSV read with `StreamReader` and no full decoded string
`UNIQUE` on `company.rfc`	**Not** added: the same RFC may identify several workspaces
Column grants on `app_user` / `SECURITY DEFINER`	Deferred to the group 5 audit hardening
Role cache	Skipped: reading membership on every request makes a revocation take effect immediately

Names and variables new: `Ingestion__MaxStatementLines`, `Ingestion__MaxDescriptionLength`, `LoginRateLimit__EmailPermitLimit`, `LoginRateLimit__IpPermitLimit`, `LoginRateLimit__WindowSeconds` and `RUN_BENCHMARKS` (section 3.1); the `importing` status, the `company_date_tolerance_max` cap and the `preserve_import_original` trigger (section 3.3); `DummyPasswordHash`, `CompanyReadOnlyException`, `UploadFailedException` and `LoginProtection` (section 3.4); status codes 409, 413 and 429 (section 3.2); and `M0005HardeningAndIndexes` (version 5).

Why this way: most of the saving comes from sending fewer commands and not repeating work; PostgreSQL's per-row cost for constraints and indexes remains. Reading membership on every request was preferred over caching roles, even at the cost of one more query.

What you learn: fewer client/server round trips do not remove PostgreSQL's per-row work (constraints and indexes), so time is still proportional to rows. A pending row is private inside its transaction, so fixing a blob once avoids re-hashing a 10 MiB original

on every update, with no new table and no deferred foreign keys. A streaming download holds a connection until the body finishes. The per-email limit can be abused to throttle a victim; the generous default and the IP limit reduce the abuse but do not eliminate it, and production needs monitoring and tuning.

Verified (PR-4 and PR-5): `dotnet build` with 0 warnings and 0 errors; `dotnet test --solution Reconciliation.slnx` passed 122 of 126 cases with four explicit skips; format verification clean. With `RUN_BENCHMARKS=1`, 123 passed and only the three future scenarios skipped. The post-refactor benchmark gave medians of 3983.3587 ms (50k lines) and 17.3551 ms (Pagos300), with samples 4143.5624 / 3983.3587 / 3926.9649 and 17.7486 / 17.3551 / 17.2354. `docker compose up --build --wait` completed with PostgreSQL, API, worker and SAT stub healthy and the migrator finished. Migrations up/down/up passed on a disposable database, preserving a synthetic original; `VersionInfo` stays at versions 1–5, the composite primary-key index of `membership` exists and all ten company tables have enabled and forced RLS. Both progress-map commands ran: 26/85 tasks, no new tasks. "Viewer cannot decide" and "Different roles" (group 8) and "Tolerance changed" (group 6) remain pending. The red commits record their expected failures before the fixes. Everything stays local; nothing was pushed. No open questions remain from this review.

PR-3: Domain model and ingestion (3.1–3.8, including 3.4b–3.4f)

What it is: the way data enters the system. It turns synthetic CFDI (XML) and reference bank CSV statements into company-isolated records with exact amounts and the original file preserved.

How it was built: each implementation follows its red-tests commit. Persistence uses real PostgreSQL through Testcontainers and the real migrator; the parsers are also tested directly with invalid and hostile inputs.

Verified: build with no warnings/errors, `dotnet test --solution Reconciliation.slnx` with 101 passing and 3 skipped, format check, Compose built and healthy, migrations up/down/up on a disposable database and both maps regenerated. Nothing was pushed.

Model, exact money and RLS (3.2, 3.8)

- **What it is:** migration `M0004IngestionTables` and the seven new tables: `import_file`, `cfdi`, `cfdi_relation`, `cfdi_payment`, `cfdi_payment_doc`, `bank_account` and `bank_movement` (see section 2.5).
- **How it works:** all carry `company_id`, enabled and forced RLS and a `tenant_isolation` policy. Foreign keys are composite, for example `(company_id, cfdi_id)`, so a row of company A cannot point at company B's invoice even by mistake. The fiscal UUID is unique per company (`UNIQUE(company_id, uuid)`). A bank movement's identity is the combination account + date + description + credit + debit + balance, and `INSERT ... ON CONFLICT DO NOTHING` turns a repeat into a "duplicate" instead of an error.
- **Names and variables:** `numeric(18,2)` for money, `numeric(20,10)` for exchange rates; `CHECK(total = subtotal - discount + transferred_tax - retained_tax + local_transferred_tax - local_retained_tax)` forces the header total to match its parts.
- **Why this way:** a `CHECK` constraint protects even against a C# code bug; PostgreSQL is the last barrier. `double` was rejected: it cannot represent 0.10 exactly. With C# `decimal` and PostgreSQL `numeric`, 0.10 + 0.20 is exactly 0.30.

- **What you learn:** composite foreign keys as an isolation technique; `GRANT UPDATE(column)` to give permission on specific columns only; `ON CONFLICT` for idempotent operations.

Safe XML and declared fiscal values (3.3, 3.4, 3.4b–3.4e)

- **What it is:** `SafeCfdiXmlReader` (safe reading) and `CfdiParser` (interpreting CFDI 4.0 and payment complement 2.0).
- **How it works:** first a bounded pass counts nodes and depth (`MaxXmlNodes`, `MaxXmlDepth`) with `DtdProcessing.Prohibit` and `XmlResolver = null`, which prevents declaring external entities (XXE) and expanding entities (billion laughs). Only if it passes is the document loaded. Then the parser validates:
  - Header arithmetic includes discounts, transferred taxes and federal and local withholdings, **without recomputing rates**: declared values are verified, tax rules are not redone.
  - The synthetic sample imports with total 1,939.27. Line-item amounts with six decimals are accepted without recomputing or storing them separately. Header, payments and balances with more than two declared decimals are rejected, including extra zeros; the MVP treats other currencies as two decimals.
  - Old payment complement versions are rejected.
  - Each payment line checks `previous balance - paid = remaining balance` and continuity between consecutive instalments present in the upload. An inconsistency is stored with `inconsistent_balance = true` and a `review_reason`, **without rejecting** the complement. Missing instalments are not inferred and the paid invoice is not required to have been imported.
  - Relation codes (even unknown ones), the global-invoice/generic-RFC indicator and issue and stamp dates are stored. Ingestion keeps relations; it does not apply them to outstanding amounts (that is group 6).
- **Names and variables:** `InvoiceUuid`, `Installment`, `PreviousBalance`, `Paid`, `RemainingBalance` are the fields of each payment line; `ReviewReason` explains why an invoice needs review.
- **Why this way:** rejected files or files unrelated to the company do not affect valid lines. Rejecting more than two declared decimals avoids silently rounding a fiscal amount.
- **What you learn:** verifying declared values is different from recomputing tax rules. Local dates carry no time zone and allow up to six fractional digits; more precision is rejected so it is not changed when stored in PostgreSQL. `LINQ to XML` (`XDocument`) and `XmlReaderSettings` are the safe way to read XML in .NET.

Reports, originals and limits (3.4f, 3.5)

- **What it is:** `CfdiImporter`, `ImportFileStore` and `MultipartUpload`: they store each file with its outcome and read the upload with limits.
- **How it works:** each file has one `import_file` row, grouped by `upload_id`, with no batch table. Status, reason and SHA-256 are stored; a rejected file keeps its hash, not its bytes. A repeated UUID points to the original (`original_file_id`) and does not replace it. A transactional lock (`pg_advisory_xact_lock`) per company/UUID serialises concurrent duplicates, backed by the unique constraint. A trigger prevents changing originals/hashes or shortening retention; there is no `DELETE`

privilege and no DELETE route. `RetentionYears` defaults to 10, is validated at start-up (minimum 5, conservative maximum 100) and sets `retained_until`.

- **Limits and decisions:** `MaxFileBytes` = 10 MiB, `MaxUploadBytes` = 20 MiB, `MaxFiles` = 20, `MaxXmlNodes` = 100000 and `MaxXmlDepth` = 64, configurable under `Ingestion`. Aggregate size or too many files reject the whole upload before anything is persisted (413/400). A file that is too large or invalid/hostile XML rejects only that file. Bytes are counted while the multipart body is read, even without `Content-Length`. The bytes of an oversize file are discarded while its hash is computed; the accepted buffer stays within the aggregate maximum. Reads and SQL receive cancellation.
- **Why this way:** a hash plus a clear rejection is preferred over storing junk. An advisory lock is simpler and cheaper than retrying after a duplicate error.
- **What you learn:** each file needs its own transaction so an invalid one does not roll back the rest; the full hash can be kept without storing the rejected content; `CancellationToken` lets work stop if the client disconnects. Full audit chaining remains in group 5.

Reference bank statements (3.1, 3.6, 3.7)

- **What it is:** `ReferenceBankStatementParser` and `BankStatementImporter`, plus the document `docs/reference/bank-csv-reference-format.md`.
- **How it works:** the exact header is `fecha,concepto,abono,cargo,saldo`; UTF-8, `yyyy-MM-dd` dates, amounts with a decimal point and two decimals. An invalid line rejects the whole file with its line number and creates no account or partial movements. An unknown bank reports `reference`. `bank=reference` and `accountReference` create or reuse the account per company/bank/label; there is no account CRUD.
- **Names and variables:** `accountReference` allows 1–40 ASCII letters/digits, spaces, hyphens and underscores; it is a label, never a full real number (suggestion: the last four digits). `RASTREO:<key>` or `REF:<number>` inside `concepto` fill `reference`; RASTREO takes precedence; with no token the reference is null. `ImportedMovements` and `DuplicateMovements` count new and repeated rows.
- **Why this way:** overlapping uploads of the same period do not duplicate movements; equal rows with a different balance are kept because they are different movements. As a conservative decision, a file with no movements is rejected and each line must have exactly one positive credit or debit. The original statement is also stored.
- **What you learn:** reusing the platform CSV reader avoids quoting and comma bugs; an interface (`IBankStatementParser`) lets real banks be added without touching the importer.

API and closing the tenancy scenarios

- **What it is:** `IngestionEndpoints`: two import `POST` routes and four `GET` query routes.
- **How it works:** every route validates membership and `X-Company-Id`. `POST /companies/{companyId}/imports/cfdi` and `/imports/bank-statement` require accountant or administrator. `GET /companies/{companyId}/imports/{uploadId}`, `/cfdi/{invoiceId}`, `/cfdi/{invoiceId}/xml`, `/imports/files/{fileId}/original` and `/banks` accept any member, including viewer. Files use the multipart field `files`.
- **Tests:** the isolation guard discovers routes, prepares successful cases, tries company B's ids under a valid company A context and compares all of B's tables

before and after. "Cross-company attempt in the test suite" now checks 404 with a real invoice of B; "Delete attempt" checks 405 and an intact invoice/original. Tasks 2.6 and 2.9 are completed.

- **What you learn:** a 404 only proves isolation after checking that the owner can in fact read the resource.

Still skipped: "Viewer cannot decide" and "Different roles" wait for real group 8 links; "Tolerance changed" waits for a group 6 run. Task 2.4 remains unticked. Imports are synchronous until the group 4 jobs.

PR-2: Tenancy and access (group 2; six tasks complete, three pending)

What it is: isolating each company from PostgreSQL, authenticating the user and letting administrators manage access and settings. Identifiers and test credentials are synthetic.

How it was built: each slice had its red-tests commit first and then the implementation commit. Testcontainers runs the real migrator against PostgreSQL 17; the application connects as `cfdi_app`, with no superuser and no `BYPASSRLS`.

Verified: `dotnet build` with no warnings or errors, `dotnet test --solution Reconciliation.slnx` with 40 passing and 5 explicitly skipped cases, `dotnet format --verify-no-changes`, Compose healthy after building and migrations up/down/up on a separate disposable database. Reverting to zero and re-applying everything also worked.

Transactions and forced RLS (2.1, 2.2)

- **What it is:** `CompanyScope` and migration `M0002TenantIsolation`.
- **How it works:** `CompanyScope.ExecuteAsync(companyId, action)` opens a connection and transaction, runs `set_config('app.company_id', ..., true)` and hands that connection to the action. In PostgreSQL, `company`, `membership` and `audit_entry` have `ENABLE` and `FORCE ROW LEVEL SECURITY` with a policy comparing `company_id` with `current_setting('app.company_id', true)`. With no context the variable is empty, `nullif(..., '')` yields NULL and no row matches: no rows are visible, and the attempt is logged. API and worker use the restricted role; the migrator uses the owner.
- **Names and variables:** `app.company_id` is the session variable; `USING` filters what can be read or modified and `WITH CHECK` filters what can be inserted; `FORCE` makes even the table owner obey the policy.
- **Why this way:** it was compared with a schema per company and a database per company (ADR-003): RLS keeps one schema and simple migrations, in exchange for requiring discipline in setting the context, which `CompanyScope` centralises.
- **What you learn:** `SET LOCAL` (or `set_config(..., true)`) ends with the transaction and avoids carrying context over when pooled connections are reused. RLS and the authenticated membership check solve different parts; both are needed.

Credentials and roles per company (2.3; 2.4 pending)

- **What it is:** `Authentication.cs`, `CompanyRoleHandler.cs`, `CompanyEndpoints.cs`.
- **How it works:** see the cycle in section 2.2. Identity's `PasswordHasher` produces PBKDF2 hashes with a random salt. If the email does not exist, login still does a slow verification against a synthetic hash so response time does not reveal which

emails exist. The fallback policy requires authentication; only health, OpenAPI and login are anonymous. The policies are `company-member` (all three roles), `accountant` (administrator and accountant) and `administrator`; they query PostgreSQL on every request.

- **Names and variables:** `sub` is the claim with the user id; `MapInboundClaims = false` stops .NET renaming claims; a `ClockSkew` of 30 s tolerates small clock differences; `ValidAlgorithms` pins HMAC-SHA256 so a token cannot declare a weak algorithm.
- **Why this way:** not storing the role in the token avoids stale permissions after a change. Cost: one query per request; accepted for simplicity.
- **What you learn:** `AuthorizationHandler<TRequirement>` and named policies; `WebApplicationFactory` lets you test a policy without publishing fake domain endpoints.

Creation, settings and archive (2.7, 2.8; 2.9 completed in PR-3)

- **What it is:** `CompanyAdminEndpoints`, `PlatformOperatorHandler`, `OperatorBootstrap` and migration `M0003`.
- **How it works:** the migrator optionally creates an operator from `OPERATOR_EMAIL` / `OPERATOR_PASSWORD`. Only that operator can create a company and its first administrator (`POST /companies`). The RFC is validated with a regular expression and a trigger (`company_rfc_immutable`) prevents changing it. A new account requires a 12-character password; an existing one keeps its password and gets the same response shape. Settings changes (`DateToleranceDays`, `AmountTolerance`, `ComplementNextBizDay`) store old and new values plus the actor in `audit_entry`, in the same transaction. Archiving (`POST /companies/{companyId}/archive`) keeps the rows.
- **Why this way:** denying INSERT on the audit table proves the settings change rolls back; the action and its audit must share a transaction. The first Compose start found that the migrator needed the ASP.NET runtime image to use `PasswordHasher`; it was fixed.
- **Demo shortcut:** sending the password at creation simplifies this demo; production would use invitation or reset links and secret management. `.env.example` contains placeholders only.
- **What you learn:** `JsonUnmappedMemberHandling.Disallow` rejects unknown fields; PostgreSQL *triggers* enforce rules no code path can bypass.

Isolation tests and ADR (2.5, 2.6)

- **What it is:** `TenantIsolationChecks`, `IsolationTests` and `docs/adr/ADR-003-tenant-isolation-rls.md`.
- **How it works:** `information_schema` discovers tables with `company_id`; RLS enabled and forced and at least one policy are required. `EndpointDataSource` discovers routes; each protected route must have a success case, an anonymous rejection and a cross-company (or non-operator) rejection. B's rows are compared before and after. Deliberately removing `membership_isolation` produced four failures, including the policy guard; restoring it brought the suite back to green.
- **Why this way:** a new table or route without an isolation test fails the build, instead of relying on someone remembering.
- **What you learn:** removing a policy causes deny-by-default; a 403 alone does not prove the policy exists, which is why the PostgreSQL structure is also verified.

Pending from this PR: task 2.4 remains unticked. Two skipped tests keep their exact names: "Viewer cannot decide" and "Different roles" wait for real group 8 links; "Tolerance changed" waits for a group 6 run. Settings persistence and audit are tested now; the behaviour of the future run is not claimed. No task is marked complete because of a skipped test.

PR-1: Foundations (tasks 1.1 to 1.5), commit `a10c3ab`

What it is: a project that builds, starts and verifies itself automatically before any business logic exists.

Verified: clean build, 4 of 4 tests, clean format check, all four services healthy in Docker, migration up/down/up on a real database and GitHub Actions run `41122200072` succeeded.

Solution and shared build configuration (task 1.1)

- **What it is:** `Reconciliation.slnx`, `Directory.Build.props`, `.editorconfig`, `global.json`.
- **How it works:** `Directory.Build.props` applies automatically to every `.csproj` below it, so one place defines how every project is built. `TreatWarningsAsErrors` together with `AnalysisLevel` makes the compiler's code analysers block sloppy code. `global.json` pins the SDK and the test runner.
- **Names and variables:** `Nullable` turns on null checking; `TreatWarningsAsErrors` turns warnings into errors; `AnalysisLevel` chooses which quality rules apply.
- **What you learn:** a *solution* (`.slnx`) groups *projects*; a *project* (`.csproj`) builds one assembly (a `.dll` or a program). Two analysers already taught something: renaming namespace `Shared` to `Common` (a reserved word in VB) and using `[LoggerMessage]` instead of string logging (faster and checked at compile time).

Core library: modules and common code (task 1.1)

- **What it is:** `src/Reconciliation.Core`, with seven folders matching the design: Ingestion, Matching, Review, Audit, Tenancy, Sat, Jobs.
- **How it works:** the API and the worker reference it, so logic is written once. Each module has a small marker class implementing `IModule` (`Common/IModule.cs`). The design rule is that modules talk through interfaces and never through another module's tables.
- **What you learn:** an `interface` is a contract with no code; a *class library* is code with no `Main`, reused by programs.

Health check and database access (task 1.2)

- **What it is:** `IDatabaseHealth`, `NpgsqlDatabaseHealth`, `HealthEndpoint`, `AddReconciliationDatabase`.
- **How it works:** `NpgsqlDatabaseHealth` runs `SELECT 1`. `MapHealth()` publishes `GET /health` and answers 200 with `{"status":"ok","database":"up"}` or 503. Docker, CI and later a load balancer use it to know whether the program is alive and reaches the database. `AddReconciliationDatabase` reads `ConnectionStrings:Default` and fails with a clear message if it is missing.
- **Names and variables:** `NpgsqlDataSource` is the shared connection pool; `ConnectionStringName = "Default"`.
- **What you learn:**

  - *Dependency injection:* `AddReconciliationDatabase` registers services once; ASP.NET hands them to whoever asks in a constructor. Tests swap the real check for a fake.
  - *Extension methods* (`this IServiceCollection services`) add methods to a type you do not own.
  - *Primary constructors* (`class NpgsqlDatabaseHealth(NpgsqlDataSource dataSource)`) and *records* (`record HealthResponse(...)`) remove boilerplate.
  - `await using`, `async`/`await` and `CancellationToken` for non-blocking database calls.
  - A *minimal API* route is simply `app.MapGet("/health", ...)`.

API host (task 1.2)

- **What it is:** `src/Reconciliation.Api/Program.cs`, the users' entry point.
- **How it works:** a few lines of *top-level statements*: build, register services, enable authentication and authorisation, map endpoints, run. `public partial class Program;` lets tests start the whole app in memory.
- **What you learn:** `WebApplication.CreateBuilder` / `Build` / `Run` is the standard ASP.NET Core lifecycle.

Worker host (task 1.2)

- **What it is:** `JobLoop`, a `BackgroundService`, for long work that must not block web requests. It can also be started several times to process jobs in parallel.
- **How it works:** the host calls `ExecuteAsync` at start-up and cancels the token on shutdown. Today the loop only logs and waits; group 4 replaces it with the real job queue.
- **What you learn:** hosted services; `partial` methods with the `[LoggerMessage]` *source generator* (the compiler writes the logging code for you).

Database migrations (task 1.5)

- **What it is:** `Reconciliation.Migrator` and `M0001CompanyUserMembership`.
- **How it works:** each migration has `Up()` (create) and `Down()` (undo). The tool runs `up` or `down` through FluentMigrator, which records applied versions in a `VersionInfo` table. It returns exit code 0 on success and non-zero on failure so scripts and CI can react.
- **Schema notes:** the RFC is validated with a `CHECK`; money-like settings use `numeric(18,2)`, never floating point; `membership` links a user to a company with a role.
- **Why this way:** database changes are versioned code, so every environment gets the same schema and any change can be undone.
- **What you learn:** *raw string literals* (`"""..."""`) hold multi-line SQL; a migration must never be edited once shared; add another one.

SAT stub (task 1.2, placeholder)

- **What it is:** a stand-in for the real SAT service so the status lookup can be built and tested offline.
- **How it works:** an empty ASP.NET app with `/health` and a 501 for everything else. Task 7.2 turns it into a real simulation.

Docker image and Compose (task 1.2)

- **What it is:** a multi-target `Dockerfile` (`api`, `worker`, `migrator`, `sat-stub`) and `docker-compose.yml`.
- **How it works:** Compose starts PostgreSQL, waits until it is healthy, runs the migrator once, then starts the rest. Containers run as a non-root user. One command (`docker compose up --build --wait`) gives anyone the complete system, identical on every machine.
- **Names and variables:** ports are published only on `127.0.0.1` (API 8080, stub 8081, database 5432); the worker exposes its health on 3001 only inside the Compose network.
- **What you learn:** *multi-stage builds* (large SDK image to build, small runtime image to run); `depends_on` with `service_healthy` and `service_completed_successfully` describe start-up order.

Tests (tasks 1.1 / 1.2)

- **What it is:** `tests/Reconciliation.Tests`, using xUnit.
- **How it works:** `[Fact]` and `[Theory]` methods; `WebApplicationFactory<Program>` runs the real API in memory with a fake database check (`HealthEndpointTests.cs`); Testcontainers starts a real PostgreSQL for the tenancy and ingestion tests.
- **What you learn:** a *fixture* (`IClassFixture`) shares costly set-up between tests; fakes keep tests fast and real databases catch SQL mistakes.

CI and ADRs (tasks 1.3, 1.4)

- **What it is:** `.github/workflows/ci.yml` and `docs/adr/ADR-001`, `ADR-002`.
- **How it works:** on every push CI runs `dotnet format --verify-no-changes`, `dotnet build`, `dotnet test`, migrations up/down/up and `docker compose up --build --wait`. The ADRs explain *why* the structure and stack were chosen (modular monolith with a worker; C# and PostgreSQL) and which alternatives were rejected.
- **What you learn:** a pipeline is just the commands you would run by hand, written down. Reading ADR-002 shows the trade-offs between TypeScript, Java, Go, PHP, Ruby and Elixir that led to C#.

Findings to remember from this PR

- A FluentMigrator dependency had a known vulnerability. Because warnings are errors, the build stopped, and the fix was to pin a patched version (`System.Security.Cryptography.Pkcs` 10.0.14) instead of silencing the warning.
- The first scaffold was TypeScript. It was dropped when the stack changed to C#; nothing depended on it.

6. Next work: group 4, Jobs

Move the synchronous ingestion services onto the PostgreSQL queue while keeping per-company transactions, per-file results and idempotency. 2.4 waits for real group 8 decisions; the settings/run test waits for group 6. The map keeps group 2 in progress on purpose.

7. The rest of the road

Group	Adds	Main types
3. Ingestion	Safe reading of CFDI XML and bank CSV, with idempotent import and document and balance checks	Module, Database
4. Jobs	PostgreSQL job queue: workers claim jobs without clashing, retries, recovery after crashes	Module, Host, Database
5. Audit	Append-only, hash-chained log of every action, with a verification function	Module, Database
6. Matching	The rule pipeline that links movements to invoices, with reasons, credit notes, cancellations and deadlines	Module
7. SAT status	The SAT client with stub, cache, retries and replay, and later the real service	Module, Host
8. Review + API	Review queue, decisions, summaries, OpenAPI document and a minimal UI	Module, API
9. Test data	Anonymiser, statement generator, edge cases, demo seed and leak scan	Tool
10. Observability	Structured logs, traces and metrics, fault-injection tests, operations guide	Pipeline, Docs
11. Delivery	Cloud design ADR, architecture views, README walkthrough, recorded walkthrough	Docs

8. How to keep this log current

1. Finish a pull request and tick its tasks in `tasks.md` (`- [x]`).
2. Run `python3 tools/progress_diagram.py` and `python3 tools/progress_diagram.py --lang es`, and commit the new `docs/diagrams/progress/status-map*` files.
3. Add a "PR-n" entry in section 5 with the same shape: What it is, How it works, Names and variables, Why this way, What you learn and Verified. If the PR introduces new environment variables, headers, columns or types, add them to section 3.
4. Update section 2 (architecture) if a flow, module or table changed.
5. Move "Next work" (section 6) to the following group.
6. Regenerate the exports in `exports/` (Word and PDF) from this file; never edit them by hand. Command: `npm install --prefix /tmp/mdtools docx` once, then `NODE_PATH=/tmp/mdtools/node_modules node tools/md_to_docx.js <log>.md exports/<log>.docx` and `soffice --headless --convert-to pdf --outdir exports exports/<log>.docx`.
7. The pull request template (`.github/pull_request_template.md`) asks for this same information, so the description can be pasted into the log.

9. Short glossary

- **Host:** a program that keeps running (web server, background worker).
- **Dependency injection (DI):** the framework creates the objects your class needs and passes them in; they are easy to replace with fakes in tests.

- **Migration:** a versioned database change with `Up` and `Down`.
- **Row-level security (RLS):** a PostgreSQL feature that filters rows by policy, enforced by the database itself.
- **JWT (JSON Web Token):** a signed credential the client presents on every request; the server checks the signature without looking up a stored session.
- **Claim:** a piece of data inside the token (here, `sub` with the user id).
- **PBKDF2:** a slow, salted password-hashing algorithm; slow on purpose to hinder brute-force attacks.
- **XXE:** an attack where XML makes the server read external files or URLs; avoided by prohibiting DTDs and resolvers.
- **Multipart:** the HTTP format for sending files and fields together in one request.
- **Idempotent:** doing the same thing twice gives the same result as doing it once (uploading the same file twice creates one record).
- **Transaction:** a set of changes that is committed whole or not applied at all; `COMMIT` / `ROLLBACK`.
- **Testcontainers:** a library that starts a real database in Docker during tests.
- **ADR:** architecture decision record; a short note on one decision and the alternatives rejected.
- **Healthy:** the container's health check passed, so services that depend on it may start.